PP

Koliko ima globalnih identifikatora u narednom primeru?

1.)

```
int f(int p) {
        int m; int n;
        return m;
}
int main() {}
```

Ima ih 2 f i main jer njihov lookup bilo gdje nece nikada biti 0 - globalno vidljivi.

2.)

```
int f() {
        int x; int y;
        return x*y;
}
```

Jedan i to je f

3.)

```
 int zero(){} int abs() {
        int x; if(x>0)
        return x; else
        return -x;
}
int main(){}
```

Ima ih 3: zero, abs i main

4.)

```
int zero(){
int abs(){
	int x; if(x > 0)
		return x;
	else
		return -x;
}
int main(){}
```

- 3

5.)

```
int f(){
	int x;
	int y;
	return x * y;
} – 1
```

6.)

```
int zero(){
	int abs(){
		int x; if(x > 0)
		return x; else
		return -x; }
} int main(){
} – 3
```

Koliko ima lokanih identifikatora u narednom primeru?

1.)

```
int f(){
        int x;
        int y;
        return x * y;
} – 2
```

2.)

```
int f() {
        int x;
        int y;
        int z;
        return x+y+z;
} -Ima ih 3 i to su x, y, z
```

3.)

```
int f() {
        int x;
        int y;
        return x*y;
} -2: x i y.
```

Koliko ima identifikatora u narednom primeru?

1.)

```
int f() {
        int x;
        int y;
        int z;
        return x + y + z;
} -3
```

2.)

```
int fun() {
        return 0;
}
int main() { }
```

Ima ih 0 jedino sto je lokalno je _TYPE a on nije _

3.)

```
int fun() {
        return 0;
}
int main() { }
```

Ima ih 0 jedino sto je lokalno je _TYPE a on nije _ID

Koliko puta skener prepozna token identifikatora (_ID) u narednom primjeru?

1.)

```
int main() {
        a=x*5;
} - 3: main, a, x;
```

2.)

```
int f(int p) {
        return p+p;
} – 4: f, p, p, p
```

3.)

```
int f() {
        int s;
        s=0;
} - 3: f, s, s
```

4.)

```
int f(int p) {
        return p+p;
} – 4: f, p, p, p
```

6.)

```
int f(){
        int a;
        a = 0; }  – 3: f, a, a
```

7.)

```
int f(int p){
        int s;
        s = s + p;
        return s;
} – 7: f, p, s, s, s, p, s
```

Kom delu miniC koda odgovara izgenerisani asemblerski kod? PP: Lokalne promjenjive su definisane sa a pa b

1.)

```
subs -8(%14), -4(%14), %0
adds -4(%14), -4(%14), %1
subs %0, %1, %0
mov %0, -4(%14)
- a = (b - a) - (a + a);
(reg0=b-a)(reg1=a+a)(reg0=reg0-reg1; reg0=(b-a)-(a+a))(a=rego0)
```

2.)

```
SUBS $2,-8(%14), %0
MOV %0,-4(%14) :
- a = 2 - b;
```

3.)

```
adds -4(%14), $3, %0
mov %0,-4(%14)
- a = a + 3;
```

4.)

SUBS -8(%14), -12(%14), %0

SUBS -4(%14), -12(%14), %1

ADDS %0, %1, %0

MOV %0, -4(%14)

- a = b - c + (a - c);

Regularni izrazi

1.) Simbol se sastoji od znaka "_" iza kojeg slijede jedno ili dva mala slova a iza nijedno ili vise velikih slova ili cifara.

_[a-z][a-z]?[A-Z]*[0-9]*

_[a-z][a-z]?([A-Z]|[0-9])*

2.) String cija je minimalna duzina 2 znaka
[a-z][0-9]?[a-zA-Z] **[a-z]{1,2}[A-Z0-9]***

4.) String cija je minimalna duzina 2 znaka?

- [a-z][0-9]?[a-zA-Z]+

5.) 0[xX][0-9A-Fa-f]{1,4}?
- 0x1234 **(tacno)**

- 0x1010 **(tacno)**

- 0x3h45 **(netacno)**

6.) [0-9]+\.[0-9]*

- 1 **(netacno)**

- 1. **(tacno)**

- 12. **(tacno)**

- 123.123 **(tacno)**
- .5 **(netacno)**

7.) (a+b?) | (cd*e)

- a, ab, aa, aab, cde, cdde **(tacno)**
- ce, aaa, abce **(netacno)**

8.) [0-9]+\.[0-9]*

- 1., 12., 123.123 **(tacno)**
- 1, .5 **(netacno)**

9.) [+,-]?[0-9]+

- +3 -2 **(tacno)**
- ,2 **(netacno)**

10.) Ako su definisana dva regularna izraza sa akcijama:

[0-9]+ { printf("broj"); }

0[xX][0-9]+ { printf("hex broj"); }

sta ce biti ispisano na izlazu:

- "hex broj"

Pisanje regularnih izraza:

- + jedan ili vise
- {3} tacno tri
- {2,5} izmedju 2 i 5
- {3,} 3 ili vise
- * nula ili vise

- ? jedan ili nijedan
- . bilo koji karakter
- \ navodi se kada mislimo na neki karakter identican nekom od specijalnih
- () grupisanje izraza
- | alterantiva jedno ili drugo
- [] navodjenje klase znakova npr [0-9]
- . bilo koji znak sem \n
- ^ negacija, sve sto nije npr. ^T sve sto nije t

Koji od navedenih pojmova se prepozna u toku parsiranja prikazanog dijela koda?

1.) x=y+5;

- num_exp **(tacno)**
- exp **(tacno)**
- variable **(netacno - variable se koristi prilikom deklaracije promjenjivih)**
- compund_statement **(netacno - blok koda u viticastim zagradama)**

2.) x=f();

-**exp , argument** (statement, assignment_statement, function_call)

3.) int main() { return a; }

- exp **(tacno)**
- return_statement **(tacno)**
- function_call **(netacno)**

4.) x=x+a;

- literal **(netacno)**
- exp **(tacno)**
- statement **(tacno)**
- variable **(netacno – kada parser prepoznaje deklaraciju prom)**

5.) x = y + 5;

- num_exp, exp

6.) x = x + a;

- literal, exp, statement

U datom pravilu meta promjenjiva X odgovara vrijednosti pojma/simbola/akcije?

1.) $3

a: _A

{printf("A");} b

{printf("B");} ;

b - jer imamo jedan izraz

2.) $4

sentence: _CAPITAL_WORD

{printf("_CAPITAL_WORD"); words_counter++;} words

{printf("words");} _DOT

{printf("_DOT");} ;

Odgovor: {printf("words");

Kakva greska postoji u miniC kodu?

1.) int x; int y; x=x+y

- **Sintaksna greska (nedostaje ;)**

2.) a = x * y – 3;

- **Leksicka (miniC ne poznaje *)**

3.)

```
int f(int p) { }
int main() { return f(); }
```

- **Semanticka (poziv funkcije f mora da ima parametar tipa int)**

4.) a=x + y ? 3;

- **Leksicka (? nije definisan)**

5.) a= x* y - 3.

- **Leksicka**

6.)

```
int main(int a){
        a = a - 5;}
```

- **Leksicka**

7.)

```
int x;
int y;
x = x + y;
```

- **Sintaksna**

8.)

```
int main() {
        int n;
        unsigned  m;
         if(n<7) return m; }
```

Semanticka (povratna vrijednost pogresnog tipa)

9.) Da li je ovaj kod semanticki ispravan?

```
int f(int num) { int n; n=10; return num + n + n; } int main() { int n; n=5; return f(n-3); }
```

Da

10.) Da li je ovaj kod semanticki ispravan?

```
int f(int num){return num;} int main() { int n; n=8; f=n+5; }
```

Ne, sa leve strane = je funkcija

Ulazni miniC kod sadrzi koji niz tokena?

1.)

```
int f() {
a = 0: }
```

_TYPE _ID _LPAREN _RPAREN _LBRACKET _ID _ASSIGN _INT_NUMBER _RBRACKET

2.) a = b + (c – d);

_ID _ASSIGN _ID _AROP LPAREN _ID _AROP _ID _RPAREN _SEMICOLON

3.)

```
int f() {
a= 0; }
```

_TYPE _ID _LPAREN _RPAREN _LBRACKET _ID _ASSIGN _INT_NUMBER _SEMICOLON _RBRACKET

4.) a = b + (c - d)

_ID _ASSIGN _ID _AROP _LPAREN _ID _AROP _ID RPAREN _SEMICOLON

5.) a = f();

_ID _ASSIGN _ID _LPAREN _RPAREN _SEMICOLON

6.) if (x == y)

_IF _LPAREN _ID _RELOP _ID _RPAREN

7.) x = x - 1

_IF _LPAREN _ID _RELOP _ID _RPAREN _ID _ASSIGN _ID _AROP _INT_NUMBER _SEMICOLON

Vrednost polja X u tabeli simbola za simbol Y

1.) Vr polja kind za simbol s?

```
int f() {
        int s;
        int t;
        return x;
}
```
- **VAR**

2.) Vr polja ptype za simbol f?

```
int f (unsigned p) {
        int n;
        unsigned m;
        n=6; }
```
- **Konstanta UINT jer je to tip parametra funkcije.**

3.) Vr polja kind za simbol a?

```
int f(){
        int x;
        int t;
        return a;
} - VAR
```

4.) Vr polja ptype(5.kolona) za simbol f?

```
int f(unsigned p){
        int n;
        unsigned m;
        n = 6;
} - UINT
```

5.) Vr polja attr za simbol b?

```
int f(){
        int a;
        int b;
        return a+b+5;
} - Attr kod promjenjivih je redni broj promjenjive pa je onda 2;
```

6.) Vr polja attr za simbol f?

```
int f(unsigned p) {
int n; unsigned m; n=6;
}
- Bude zapisana vrijednost 1 jer attr sazdrzi broj parametara funkcije
```

7.) Vr polja attr za simbol m?

int f(unsigned p) {

int n; unsigned m; n=6;

} **- 2 (druga promjenjiva po redu)**

8.) Vr polja attribute za simbol f?

Za ulaz:

int f(){ int b;

return b + 2 - f();

} - **2**

9.) Vr polja atr1(4. kolona) za simbol m?

int main(){ int n; unsigned m; n = 6;

} – **2**

Odrediti ispravan X skup pojma Y

Primer za FIRST skup

expression -> term expression1

expression1 -> "+" term expression1 |epsilon

term -> factor term1

term1 -> "*" factor term1 | epsilon

factor -> "(" expression ")"

|id

Primer za FOLLOW skup

expression -> term expression1

expression1 -> "+" term expression1 |epsilon

term -> factor term1

term1 -> "*" factor term1 | epsilon

factor -> "(" expression ")" |id

FOLLOW(expression) = FOLLOW(expression1) = {), $ }

FOLLOW(term) = FOLLOW(term1) = { +,), $ }

FOLLOW(factor) = { *, +,), $ }

1.) FIRST skup pojma X

E -> T X

T -> (E) | int Y

X -> + E | epsilon

Y -> * T | epsilon

{ + , epsilon }

2.) FOLLOW skup pojma T

E -> T X

T -> (E) | int Y

X -> + E | epsilon

Y -> * T | epsilon

{ +, $,)}

Odrediti koliko ima baznih blokova u delu asemblerskog koda:

1.)

```
@switch
        JMP @test0 1
@case0_0
        MOV $1,state 2
        JMP @exit0 3
@case0_1
        MOV $2,state 4
        JMP @exit0 5
@default
        MOV $0,state 6
        JMP ...
```

- **7**

Algoritam: Prva instrukcija je pocetak bloka Svaka instrukcija koja je cilj nekog skoka je pocetak bloka, svaka instrukcija nakon skoka je pocetak bloka

Odrediti na kojoj liniji se zavrsava X bazni blok u delu asemblerskog koda ?

1.) Drugi

```
1. MOV $0, suma
2. MOV $0, i
@lab3:
3. CMPS i, $5
4. JGTS @exit3      ; ← Kraj baznog bloka jer je ovo skok koji završava blok
5. ADDS suma, i, %0
6. MOV %0, suma
```

- **4**

2.) cetvrti

@do0:

1. ADDS a, $1, %0

2. MOV %0, a

@if7

3. CMPS c, d

4. JLES %false7 ; ← Prvi skok koji završava bazni blok

5. @true7 MOV $1, c

6. JMP @exit7 ; ← Skok koji završava bazni blok

7. @false7 MOV $2, c

- **7 (@false7 početak novog bloka, odnosno blok pre @false7 završava na liniji 6)**

Dat je izraz u postfiksnoj notaciji.

Tom izrazu odgovara sledeci izraz u infiksnoj notaciji

1.) 2 a b + unarani_minus * c d - /

- **(2*(-(a+b)))/(c-d)**

2.) (a+b)/(-5*(-3+c))

- **a b + 5 unarni_minus 3 unarani_minus c + * /**

3.) (a-b)/(-(8+d)-a)*c

- **a b - 8 d + unarni_minus a - / c ***

4.) (a-b)/(-(-8+d)-a)*c

- **a b - 8 d + unarni_minus a - / c ***

5.) -(c+d)*((c-7)+a)/b

- **c d + unarni_minus c 7 - a + * b /**

Odrediti na kojoj liniji se nalazi nedostupna naredba u asemblerskoj sekvenci:

1.)

```
@lab0:    ADDS %4, %1, %1      ; 1
          DIVS %2, $5, %2      ; 2
          JMP @lab1            ; 3
          MULS %3, $7, %3      ; 4 ← NEDOSTUPNA
@lab1:    MOV -4(%14), -8(%14) ; 5
```

2.)

```
@lab0:    CMPS c, d            ; 1
          JE @true0            ; 2
          JNE @exit0           ; 3
          SUBS %3, $1, %3      ; 4 ← NEDOSTUPNA
@true0:   MOV c, d             ; 5
```

Odrediti u kojoj liniji se nalazi suvisna naredba u asemblerskoj sekvenci:

1.)

```
1. @while:  CMPU a, b
2.          JEQ @false1
3. @true0: CMPU a, b     ← Suvišna naredba (već poređeno u liniji 1)
4.          JLEU @false1
5. @true1: SUBU a, b, a
6.          JMP @exit1
```

7. @false1: SUBU b, a, b

8. @exit1: ADDU a, b, c...

- **3**

2.)

1. MULU %1, c, %1

2. MOV c, a

3. SUBU b, a, b

4. DIVU a, b, %2

5. MOV c, a ← Suvišna naredba (ponavlja liniju 2, a bez promene između)

6. ADDU a, %2, %2

- **5**

3.)

1. MOV $5, %2

2. ADDS $1, %2, %2

3. MOV %2, a

4. MULS a, $5, %3

5. MOV %3, b

6. MOV a, %2 ← Suvišna naredba (ništa se novo ne dešava, a već je `a = %2`)

7. ADDS ...

- **6**

4.)

1. @while0: CMPU a, b

2. JEQ @false1

3. @true0: CMPU a, b ← Suvišna naredba

4. JLEU @false1

5. @true1: SUBU a, b, a

6. JMP @exit1

7. @false1: SUBU b, a, b

8. @exit1: ADDU a, b, c

- **3**

Odrediti u kojoj liniji je moguca algebarska transformacija u asemblerskoj sekvenci ?

- Neke instrukcije se mogu uprostiti (ovo se zove *redukcija snage*, engl. *strength reduction*):

```
x := x * 0          x := 0
y := y ^ 2          y := y * y
y := 2 * x          y := x + x
x := x * 8          x := x << 3
x := x * 15         t := x << 4; x := t - x;
```

1.)

CALL pp

MOV %3,b

MOV c,a

MULS $2,%2,%3 // OVA LINIJA

CMPS a,b

JGTS @lab1

- **MULS $2, %2, %3 → može se zameniti sa ADDS %2, %2, %3**

Koja optimizacija moze da bude/je primenjena na naredbi?

1.)

t=1+2;

Slaganje konstanti

2.)

a = 2; a = 2;
b = 4 + 3; b = 7;

-**slaganje konstanti**

jer je a+a =4 a ovde je u prekompajliranju to sracunato i odma zapisano

3.)

k = 2; k = 2;

r = r * 4; => r = r << 2

k = r + 3' k = r + 3;

-**algebarska transformacija**

4.)

x = 6; x = 6;

y = 4; => y = 4;

z = x * y; => z = 6 * 4;

-**slaganje konstanti**

5.)

p = 4; p = 4;

p = p + 0; => t = p + 3;

t = p + 3;

- **algebarska transformacija**

Java

1.) a = -b * (c + d) / (e - f);

```
BIPUSH 0      ; ubacuje konstantu 0 na stek
ILOAD b       ; učitava vrednost promenljive b
ISUB          ; računa 0 - b (tj. -b)
ILOAD c       ; učitava vrednost promenljive c
ILOAD d       ; učitava vrednost promenljive d
IADD          ; računa c + d
IMUL          ; množi (-b) * (c + d)
ILOAD e       ; učitava vrednost promenljive e
ILOAD f       ; učitava vrednost promenljive f
ISUB          ; računa e - f
IDIV          ; deli prethodni rezultat sa (e - f)
ISTORE a      ; smešta rezultat u promenljivu a
```

2.) a = b + c;

Kom iskazu odgovara prikazani Java asemblerski program?

```
ILOAD b
ILOAD c
IADD
ISTORE a
```

Random zadaci

1.) Troadresni kod za izraz:

a = b * c + b * d;

Troadresni kod (operator, operand1, operand2, result):

_t1 = b * c;

_t2 = b * d;

_t3 = _t1 + _t2;

a = _t3;

2.) Asemblerski kod za slog strukture:

```
struct {
    int x;
    int y;
} z;
z.y = z.x;
```

MOV $z, %0 — Učitaj adresu promenljive z u registar %0.

MOV 0(%0), 4(%0) — Prekopiraj sadržaj sa adrese z.x na adresu z.y

3.) Asemblerski kod za niz:

```
int n[10];
n[0] = n[1];
```

MOV $n, %0 — Učitaj adresu početka niza n u registar %0.

MOV 4(%0), 0(%0) — Kopiraj sadržaj sa n[1] (offset 4 bajta) u n[0] (offset 0).

Teorija

1. Najviše kompajlera radi:

lokalnu optimizaciju

2. Prilikom generisanja koda, najčešće korišćene vrednosti se smeštaju:

u registrima

3. Kod dvoprolaznog kompajlera prvi prolaz obuhvata:

prednji modul kompajlera (front end)

4. Kod dvoprolaznog kompajlera drugi prolaz obuhvata:

zadnji modul kompajlera (back end)

5. Optimizacija koja se vrši na jednom baznom bloku je:

lokalna

6. Optimizacija petlji ima za cilj da:

smanji broj naredbi u telu petlje

7. Šta se u toku optimizacije međukoda radi sa suvišnim naredbama?

izbacuju se

8. Bootstraping je proces u kom kompajler:

kompajlira sam sebe

9. Interpreter sadrži:

leksičku, sintaksnu, semantičku analizu, generisanje i optimizaciju međukoda

10. U hipotetskom asembleru, kao pokazivač frejma se koristi registar:

%14

11. Koliko ukupno registara ima hipotetski asembler?

16

12. Registar %15 se koristi kao:

pokazivač vrha steak

13. Poželjne osobine kompajlera su:

dobra dijagnostika grešaka i dobra optimizacija

14. Koje faze kompajliranja pripadaju zadnjem modulu kompajlera (back end)?

generisanje koda i optimizacija koda

15. Koje faze pripadaju prednjem modulu kompajlera (front end)?

leksička, sintaksna i semantička analiza

16. Koji parseri su pogodni za automatsko generisanje?

LR parseri

17. Kompajler za programski jezik C za Intel procesor ima prednji modul PM1 i zadnji modul ZM1. Kompajler za C++ za MIPS procesor ima prednji modul PM3 i zadnji modul ZM3. Ako svi koriste isti međujezik, PM1 i ZM3 odgovaraju kompajleru:

programskog jezika C za MIPS procesor

18. Izgled organizacije memorije za pokrenuti program (runtime), od nižih ka višim adresama:

1. code
2. static data
3. stack
4. heap

19. Vrste optimizacije:

- algebarske transformacije
- slaganje konstanti
- nedostupan kod
- mrtav kod
- suvišan kod

20. Za lokalne identifikatore u miniC programu važi da:

- postoje samo za vreme izvršavanja funkcije
- oslobađaju se na kraju izvršavanja funkcije

21. Sintaksnu grešku predstavlja:

pogrešan redosled simbola

22. Pretraga tabele simbola u cilju pronalaženja identifikatora se vrši pomoću funkcije:

lookup_symbol()

23. Regularna gramatika se koristi za:

pisanje leksičkih pravila

24. Peephole optimizacija posmatra:

sekvencu uzastopnih naredbi

25. Kros kompajler generiše kod za:

računar drugačije arhitekture

26. Međujezik:

- ima manje detalja od ciljnog jezika (više je apstraktan)
- ima više detalja od izvornog jezika (manje je apstraktan)

27. Šta se u toku optimizacije međukoda radi sa naredbama suvišnih skokova?

modifikuju se

28. Labele počinju malim slovom, nakon čega slede slova, cifre ili donja crta. Iza labele se stavlja dvotačka. Sistemske labele počinju znakom @.

29. Operandi mogu biti:

- neposredni
- registarski

- direktni
- indirektni
- indeksni

30. Naredbe u asembleru:

- poređenje: CMPX ulazni operand, ulazni operand
- skok: JMPx ulazni operand
- uslovni skok: JEQ ulazni operand
- rukovanje stekom: PUSH, POP
- poziv potprograma: CALL ulazni operand
- povratak iz podprograma: RET
- sabiranje: ADDx ulazni operand, ulazni operand, izlazni operand
- oduzimanje: SUBBx ulazni operand, ulazni operand, izlazni operand
- množenje: MULx ulazni operand, ulazni operand, izlazni operand
- deljenje: DIVx ulazni operand, ulazni operand, izlazni operand
- prebacivanje: MOV ulazni operand, izlazni operand
- konverzije: TOF (int u float), TOI (float u int)
- zauzimanje memorijskih lokacija: WORD broj

31. Registar za povratnu vrednost funkcije, pokazivač frejma i pokazivač vrha steka su:

%13, %14, %15

32. Registri i memorijske lokacije zauzimaju:

po 4 bajta

33. Status bitova registra menja naredba:

CMP

34. Prethodni pokazivač frejma se stavlja:

posle povratne adrese

35. Na stack frame-u, od viših ka nižim adresama nalaze se:

argumenti, povratna adresa, prethodni pokazivač frejma, lokalne promenljive, pokazivač steak

36. Zadatak parsera je:

- proveriti da li je ulazni niz simbola u skladu sa zadatom gramatikom
- proveriti semantiku programa

37. Regularni izrazi služe za opis:

simbola

38. BNF notacija služi za opis:

sintakse

39. Globalna optimizacija analizira:

dijagram toka programa

40. Semantika programskog jezika se opisuje:

neformalno